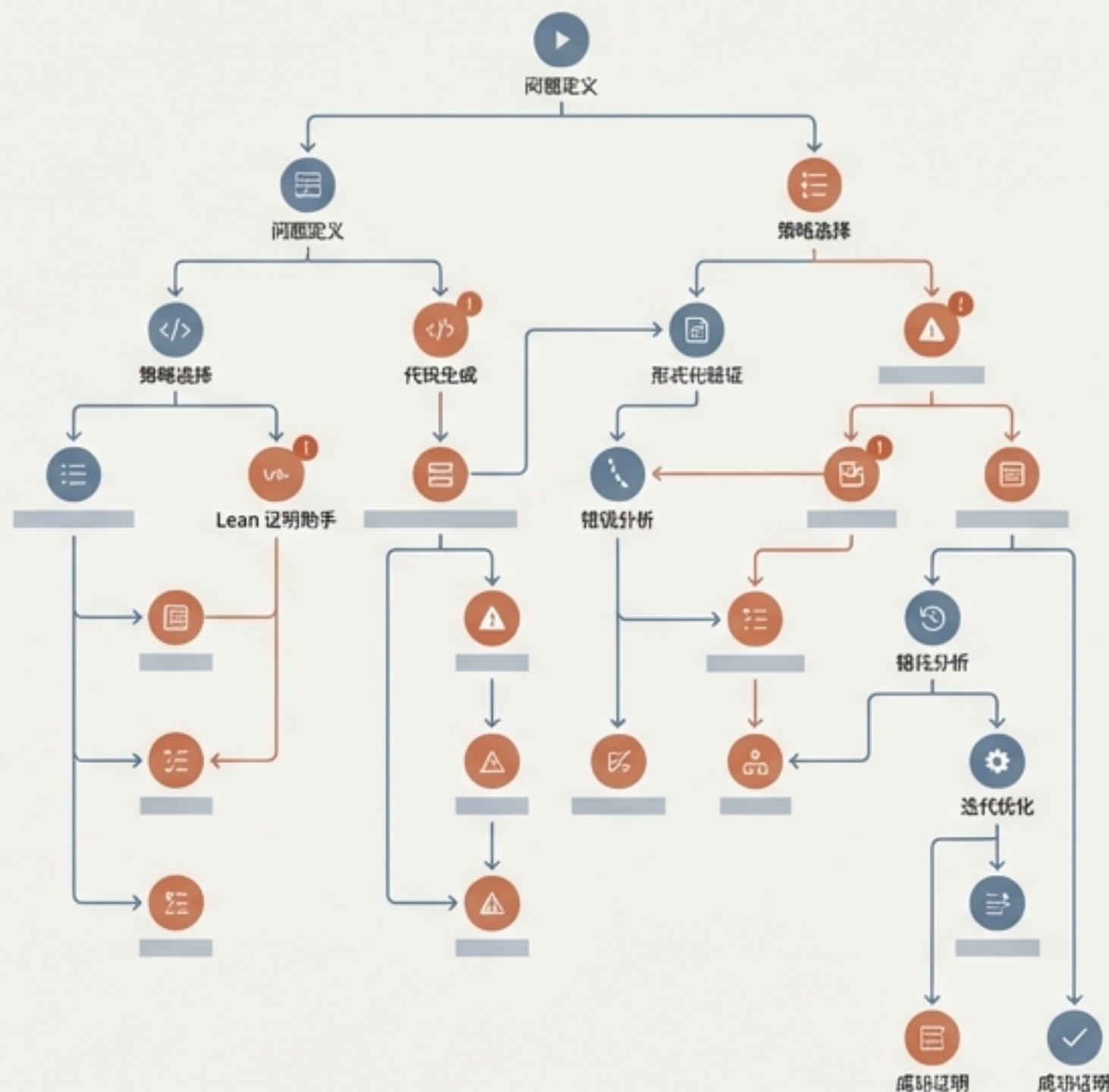


与 Claude Code 结对编程

陶哲轩关于 Lean 数学证明的
AI 形式化实践实录

1689 $\Rightarrow$ 2

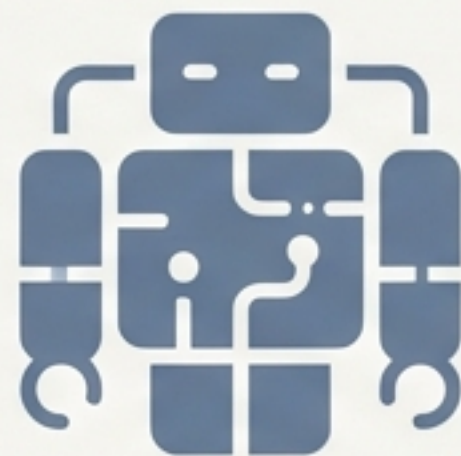


跨越九个月的工具演进



九个月前

- GitHub Copilot + 手动步进
- 需要大量人工干预与微小的 AI 辅助



现在

- Claude Code 全自动智能体
- 目标：测试智能体能否独立执行形式化证明

全自动化的陷阱与代价

指令：一步到位 (Do the whole thing)



运算 45 分钟 → 耗尽 Token



导致电脑死机

面对复杂的数学逻辑，缺乏宏观规划的 AI 会陷入过度思考，最终产出为零。

破局策略：基于配方的分步指令

核心准则：不要让 AI 自行发挥，而是强迫其与人类的“非形式化证明”严格对齐。

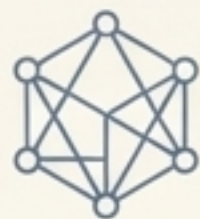
Step 0: 定义符号 (S & F notations)

Step 1: 构建骨架 (Skeleton proof)

Step 2: 逐行翻译 (Line-by-line translation)

阻断过早优化：引入占位符

- 快速确立 S 和 F 的符号定义
- 要求 AI 生成包含 Lemma 1, 2, 3 的骨架证明
- 所有具体的证明步骤全部使用 sorry 占位



防止 AI 过早陷入细节推导、反复犯错并生成无效缓存。先搭框架，后填血肉。

```
lemma 1 :  
  sorry
```

```
lemma 2 :  
  sorry
```

```
lemma 3 :  
  sorry
```

非形式化到形式化的逐行映射



强制映射：

将非形式化证明的每一行转化为独立的 Lean 代码行。

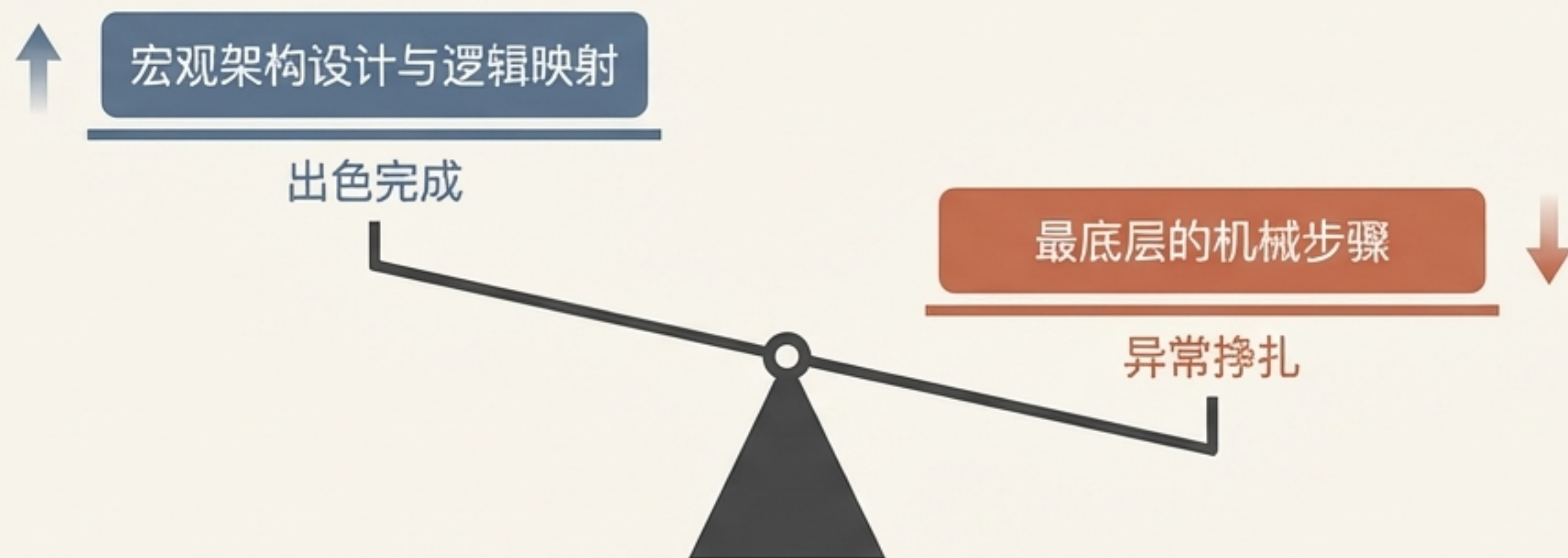
避免失控：

防止 AI 抛弃人类的有效证明逻辑，自创无效证明。

高可维护性：

使得最终代码极具可读性，大幅降低调试难度。

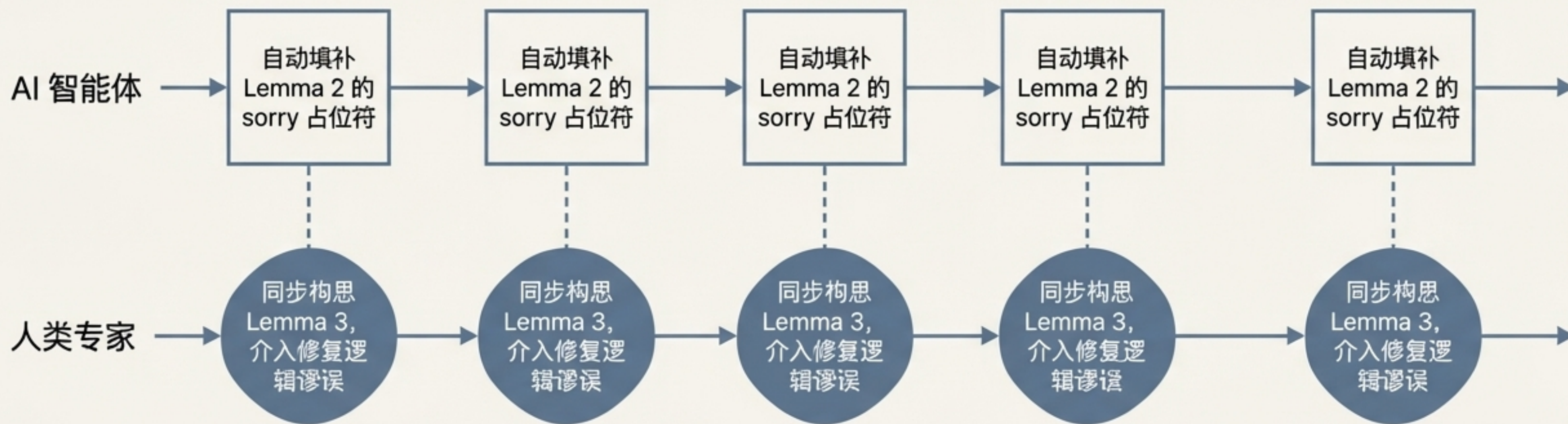
智能体的反常盲区



表现症状：

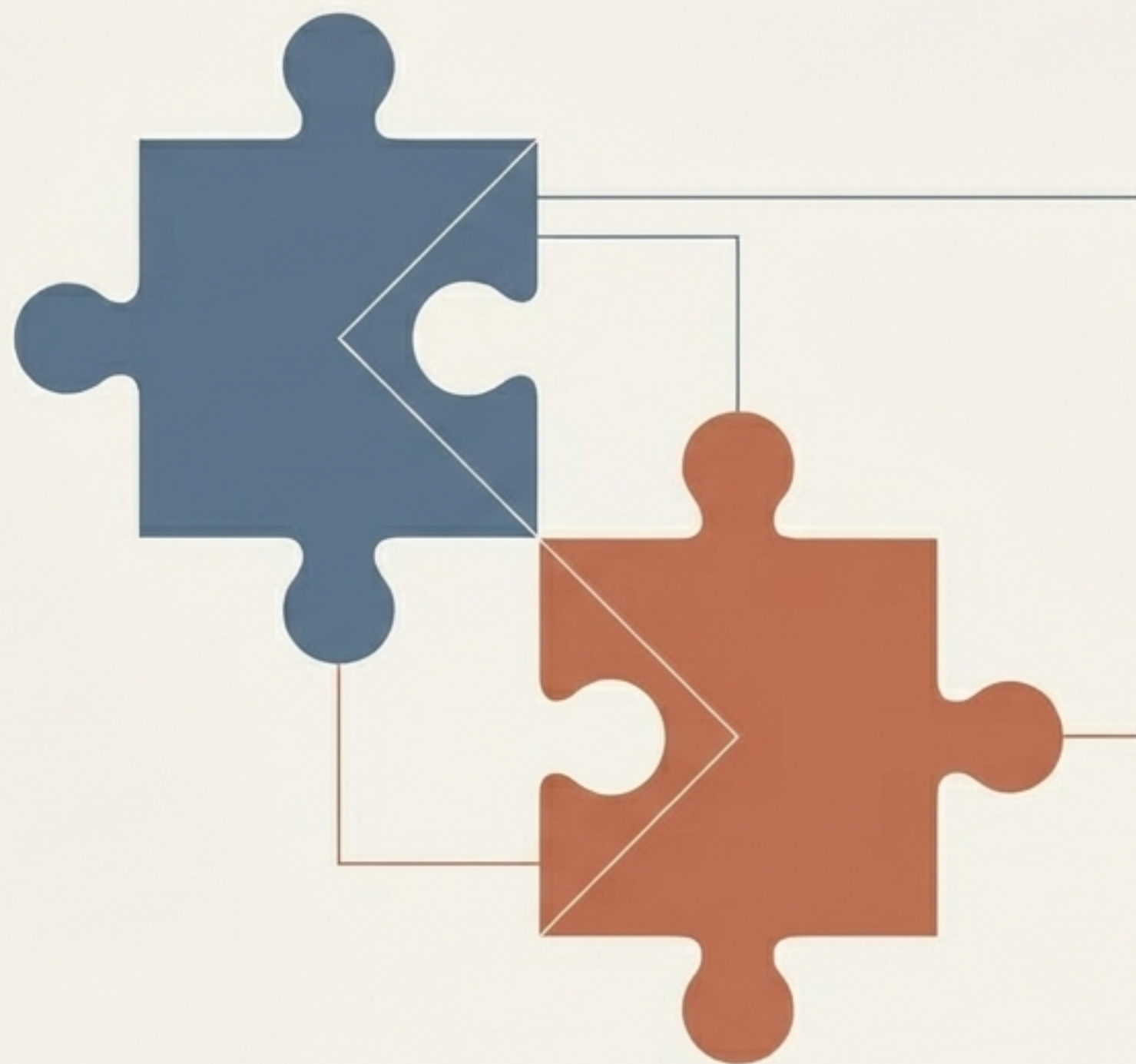
- 消耗大量 Token
- 频繁触发逻辑回溯
- 过度复杂化简单问题

终极形态：人机并行 workflow



当前 AI 辅助数学研究的最佳用例：人机分工，并行推进。

系统韧性与动态重构



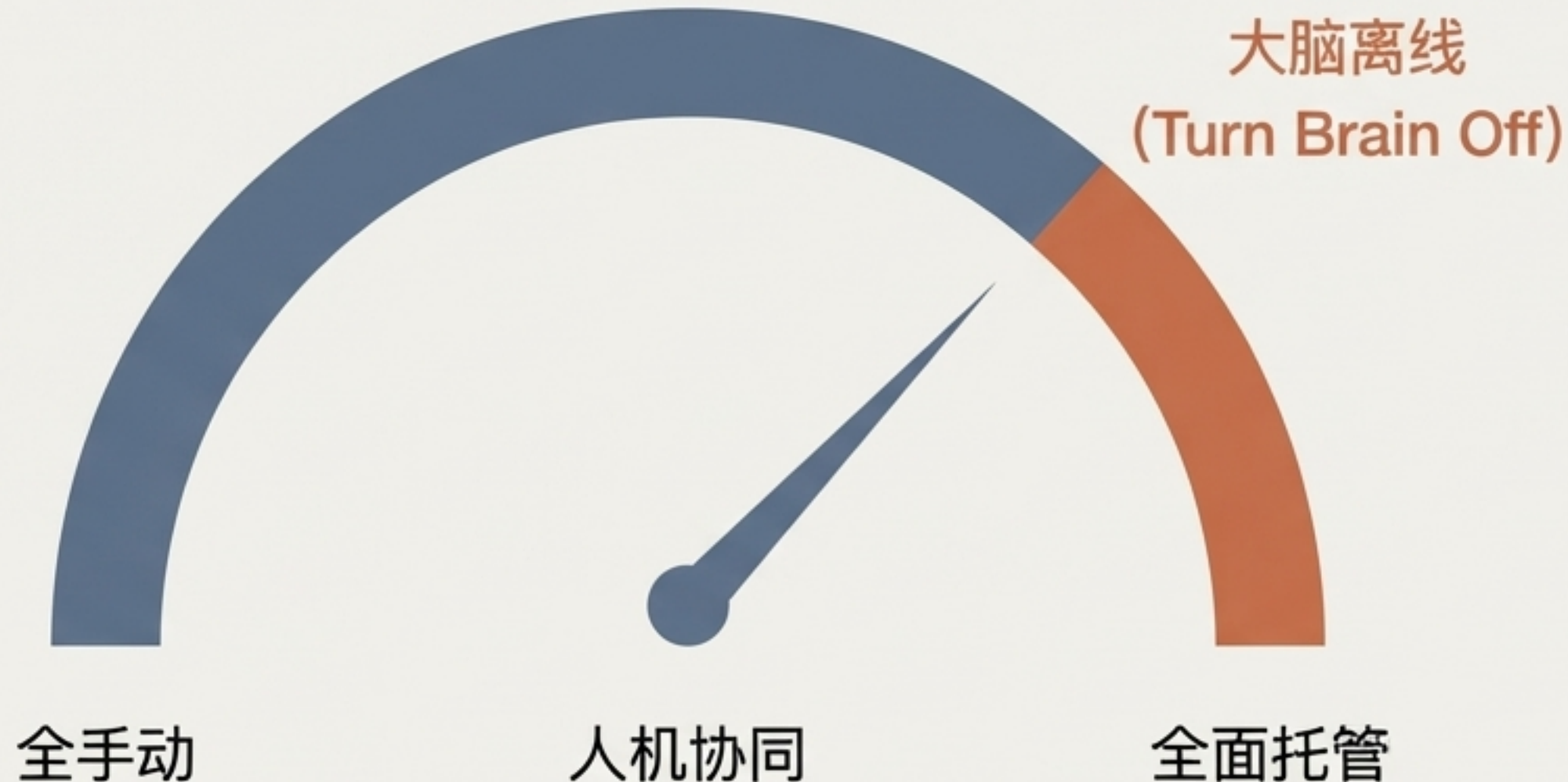
系统韧性

实验中途遭遇系统崩溃，但分步式工作流使进度得以无损恢复。

动态重构

人类介入提取独立引理以优化代码结构，Claude 在人类编辑时展现出极强适应性，无缝接手后续修改。

自动化的诱惑与理解力的底线



全面托管极其诱人，但一旦过度依赖工具且系统报错，若对证明过程缺乏理解，将陷入束手无策的境地。

结论：人类必须保持在场（Keep involved），持续审核代码逻辑。

结果与核心启示

耗时

约 30 分钟（包含崩溃恢复）

结果

完成全套编译，零报错

关键

指令明确性 > AI 原始算力

“选择让你感到舒适的自动化水平。”

—— 陶哲轩